NikPak

User Manual and Technical Guide

Version	1.13.2
Language	Deutsch (de)
Date	2026-11-14

Contents

Teil I · Benutzerhandbuch 3
1. Was NikPak ist 4
2. Installation und das erste Archiv 5
3. Kompressionsstufen und Geschwindigkeit 7
4. Passwörter und Verschlüsselung 8
5. Geteilte Volumes und selbstextrahierende Archive 9
6. Cover, rechtliche Erklärungen und Pakete 10
7. unNik, der kostenlose Browser 11
8. Skins, Sprachen und Registrierung 12

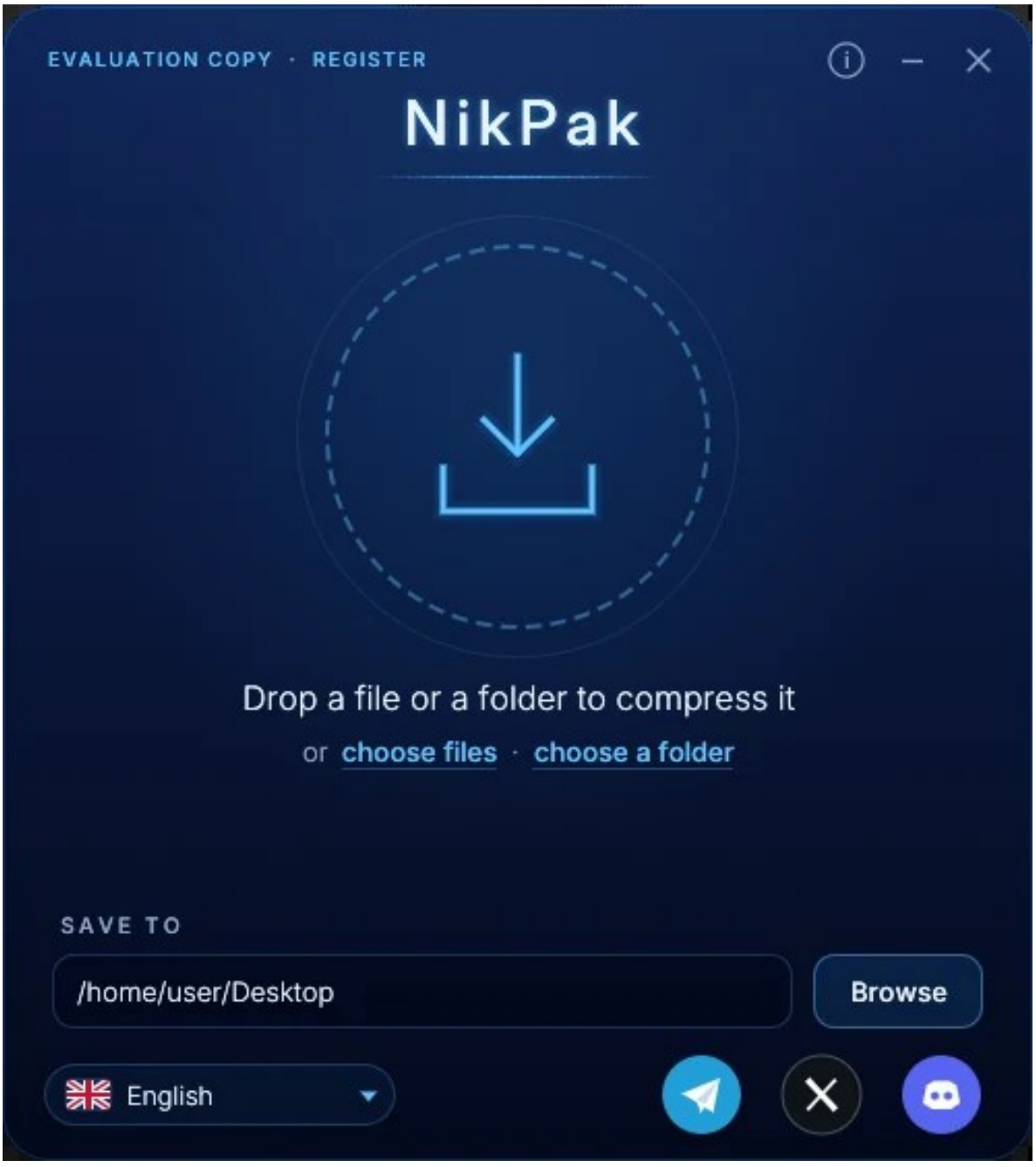
Teil II · Technischer Leitfaden 14
1. Architektur 15
2. Das .nik-Format 15
3. Die Kompressions-Engine 16
4. Verschlüsselung und warum sie quantenresistent ist 16
5. Benchmarks, Härtung und Build 17
6. Dieses Buch 18

Teil I · Benutzerhandbuch

Alles, was NikEdit tut, Fenster für Fenster.

1. Was NikPak ist

NikPak ist ein schneller, verschlüsselnder Archiver für das **.nik**-Format, und **unNik** ist der kostenlose Archivbrowser, der diese Archive auf jedem Rechner öffnet. Zusammen ersetzen sie das gewöhnliche ZIP an den zwei Stellen, an denen es am schwächsten ist: Größe und Vertrauen.



Das NikPak-Fenster, bereit zum Ablegen von Dateien.

Es komprimiert mit **Zstandard**, **LZMA2** und **PPMd**, rekomprimiert das **Deflate** in Office-Dateien und PDFs bitgenau, verschlüsselt mit **Argon2id + XChaCha20-Poly1305** und kleidet alles in ein Fenster, das sich skinnen lässt. Dateien werden auf allen Kernen komprimiert, ein Archiv kann in Volumes aufgeteilt oder in eine selbstextrahierende `.exe` verwandelt werden, und es können ein Titelbild und eine rechtliche Erklärung beigelegt werden, die jeder vor dem Entpacken akzeptieren muss.

Die beiden Programme

- **nikpak** — der Archiver: ein Fenster, auf das Sie Dateien ziehen, und eine Kommandozeile mit Syntax im RAR-Stil (`nikpak a -p secret.nik Photos\`). Shareware: kostenlos für Privatpersonen, lizenziert pro Arbeitsplatz für Unternehmen.
- **unNik** — der kostenlose Archivbrowser: öffnet `.nik`-Archive in einem Fenster, durchsucht Ordner, sucht, zeigt Text an und entpackt. Kostenlos für alle, für immer, ohne Registrierung.

Beide sind in Rust geschrieben und laufen unter Windows, macOS und Linux. Das `.nik`-Format ist offen und in Teil II dokumentiert, sodass ein heute geschriebenes Archiv von einem morgen geschriebenen Programm gelesen werden kann.

Was es Ihnen bringt

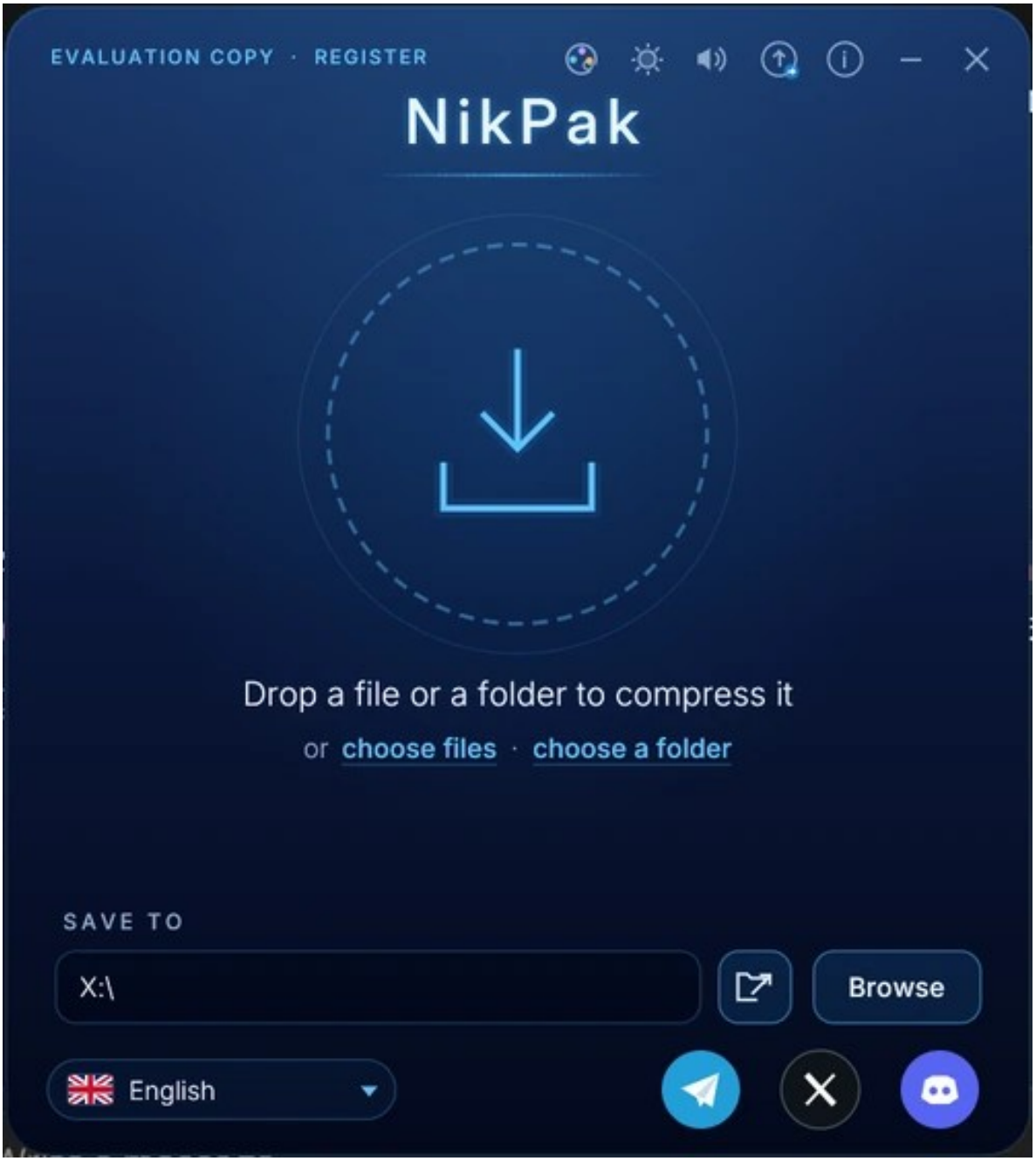
- **Kleinere Archive.** Auf der höchsten Stufe gehen Binärdaten durch LZMA2 und Text durch PPMd, jeweils pro Block gewählt — 38,8 % kleiner als RAR 7 bei Fotos, 12–15 % bei Audio.
- **Echte Verschlüsselung.** Schlüsselableitung mit Argon2id und XChaCha20-Poly1305; auch die Dateinamen werden verschlüsselt, und ein falsches Passwort oder ein manipuliertes Archiv wird erkannt, statt still halb entpackt zu werden.
- **Mehrkern.** `-m5` teilt das Archiv in ausgewogene Blöcke auf und komprimiert jeden in einem eigenen Thread; `-mt<N>` ist der Regler.
- **Verlustfreie Medienfilter.** WAV, AIFF, MOD und BMP werden mit eigens dafür gebauten Filtern probeweise komprimiert, und das kleinste Ergebnis bleibt.
- **Pakete.** Packen Sie Dateien in ein verschlüsseltes, verkäufliches `.nik`-Paket mit einem Namen, einer Beschreibung, einem quadratischen Cover und einer zu bestätigenden Lizenz.

2. Installation und das erste Archiv

Führen Sie das Installationsprogramm aus, oder verwenden Sie die portable Programmdatei. Das Installationsprogramm fügt `nikpak` und `unNik` dem Anwendungsmenü und der Kommandozeile hinzu. Unter macOS und Linux kopieren Sie beide Binärdateien nach `/usr/local/bin`.

Über das Fenster

Starten Sie NikPak und **ziehen Sie Dateien oder Ordner auf das Fenster**. Das Ablagefeld fragt, wohin das Archiv geschrieben werden soll und mit welchen Einstellungen; das Archiv wird dann im Fenster aufgebaut, mit einem Fortschrittsbalken pro Datei und für das Ganze. Das Ablegen eines Archivs bewirkt das Gegenteil — es öffnet das Entpacken-Feld.



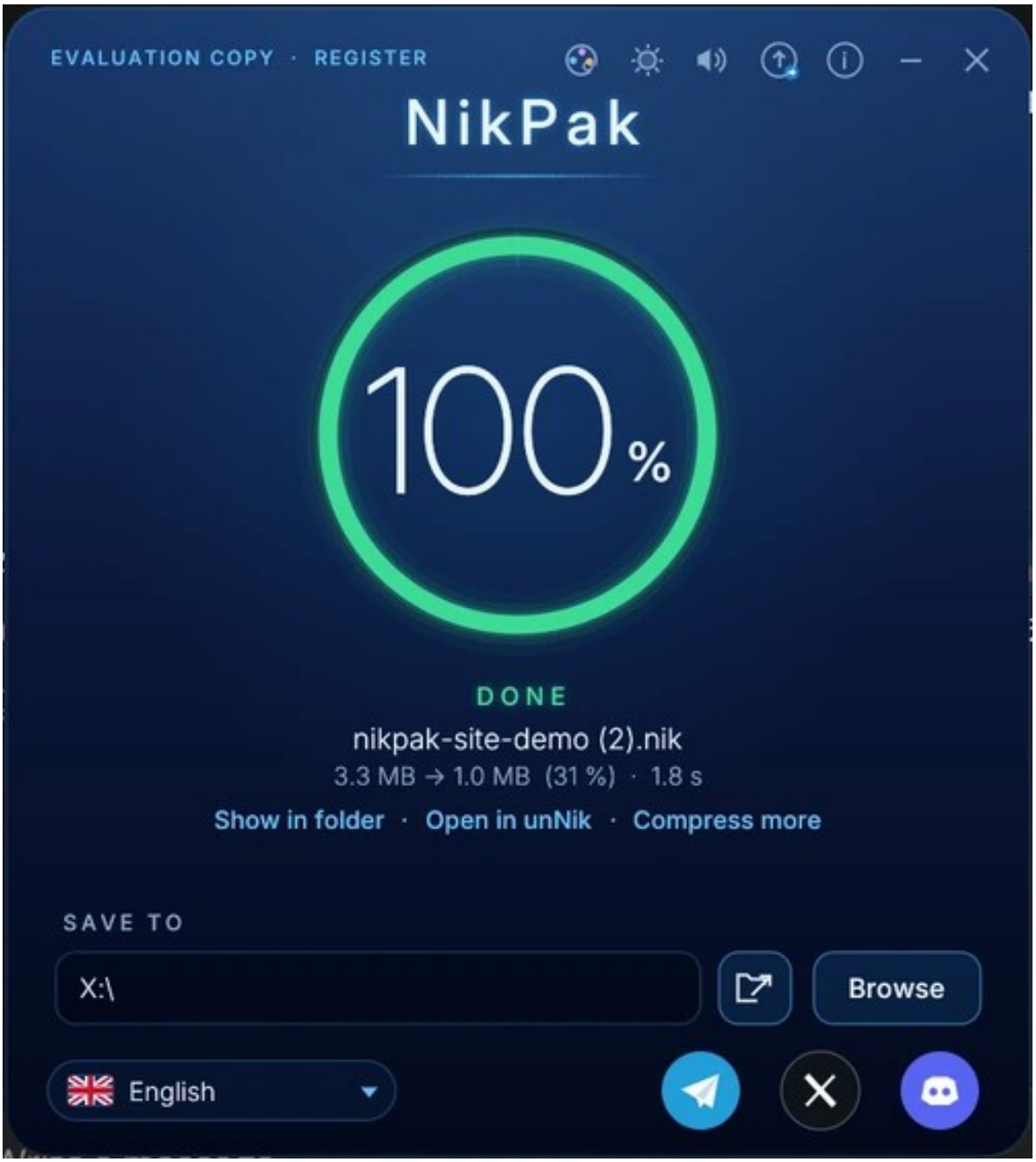
Das Ablagefeld: was gepackt wird, wohin es geschrieben wird und die Stufe.

Über die Kommandozeile

```
nikpak a backup.nik C:\Docs\*.txt Photos\        # create an archive
nikpak a -m5 -mt8 big.nik Projects\             # maximum compression, 8 threads
nikpak a -p secret.nik Taxes2026\               # encrypted, asks for the password
nikpak x backup.nik C:\Restore\                # extract with paths
nikpak l backup.nik                             # list
nikpak t secret.nik                             # test (asks for the password)
```

`nikpak -help` gibt die vollständige Referenz aus, einschließlich der Optionen, die dieses Kapitel nicht verwendet. Für Server gibt es einen Build nur für die Kommandozeile (ohne Fenster).

3. Kompressionsstufen und Geschwindigkeit



Das Kompressionsfeld: Stufe, Threads und die Schätzung.

Sechs Stufen, von **Store** bis **Maximum**. Die Voreinstellung ist das Maximum, weil ein Archiv gewöhnlich einmal geschrieben und oft gelesen wird — aber eine Stufe ist eine Wahl, kein Gesetz, und die Schätzung im Feld sagt Ihnen, was sie kostet, bevor Sie sich festlegen.

Die Stufen

Stufe	Was sie tut	Wann sie sich lohnt
`Store`	Keine Kompression, nur der Container und die Verschlüsselung	Bereits komprimierte Medien, die Sie in einer Datei haben möchten
`Fastest` / `Fast`	Mehrthreadiges Zstandard	Riesige Ordner, interaktive Arbeit

Stufe	Was sie tut	Wann sie sich lohnt
`Medium`	Zstandard mit größerem Fenster	Die Wahl für jeden Tag
`High`	LZMA2 mit PPMd für Text	Dokumente, Quellcode, Protokolle
`Maximum`	LZMA2 / PPMd pro Block, Medienfilter, Deflate-Rekompression	Archive, die Sie behalten und versenden

Threads ist `-mt<N>`, oder *Automatisch* im Fenster. `-m5` teilt das Archiv bereits in ausgewogene Blöcke auf und komprimiert jeden in einem eigenen Thread; die Arbeit pro Datei — Medienproben, Deflate-Rekompression — läuft auf jeder Stufe in einer Pipeline.

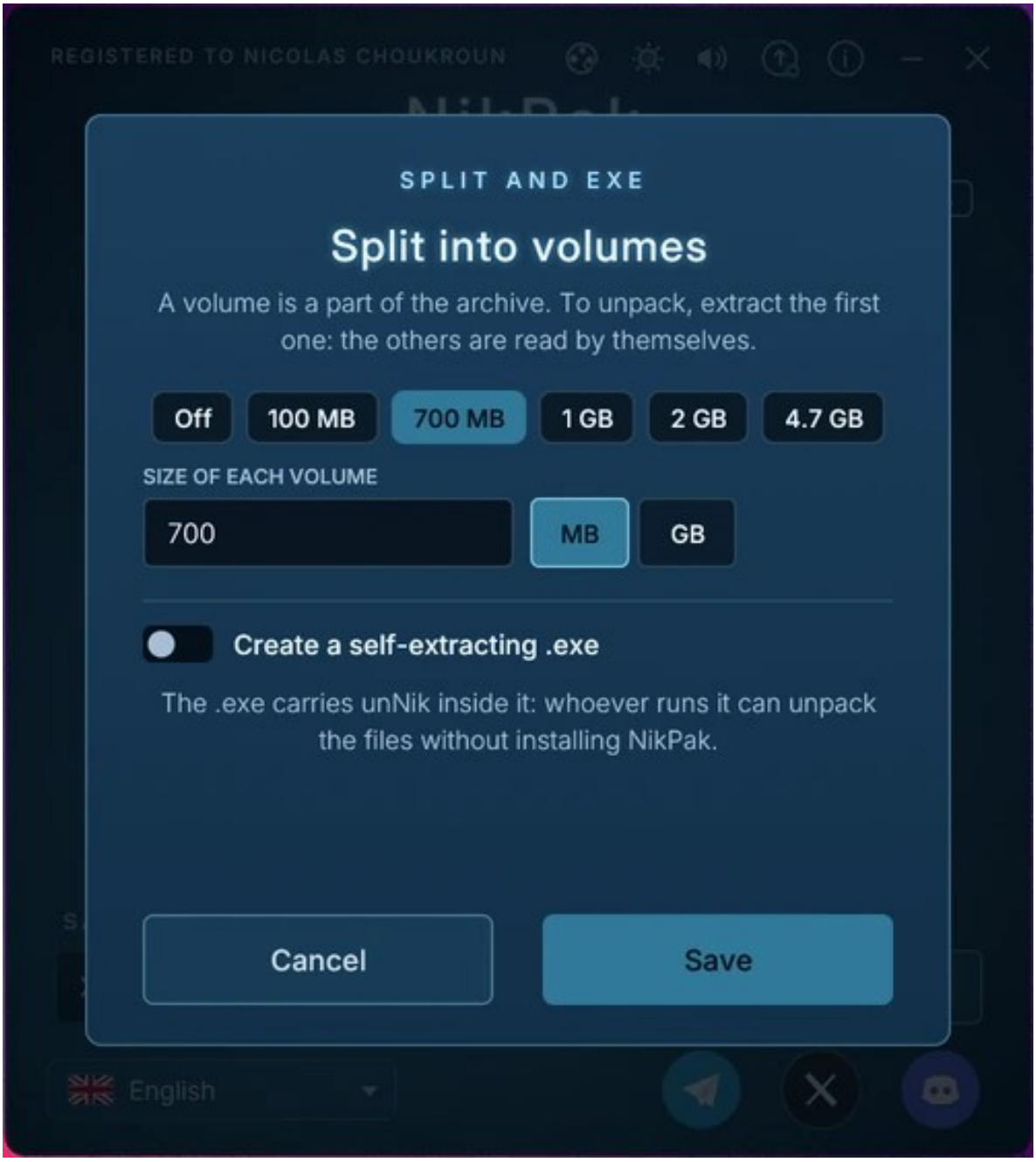
> Office-Dokumente (`docx`, `xlsx`, `pptx` …), PDFs und `.gz`-Dateien sind bereits intern Deflate-komprimiert. NikPak dekodiert diese inneren Streams, komprimiert den Klartext-Inhalt deutlich besser und baut die ursprünglichen Streams beim Entpacken **Bit für Bit** wieder auf — das Archiv ist kleiner, und die Datei, die Sie zurückbekommen, ist die Datei, die Sie hineingegeben haben.

4. Passwörter und Verschlüsselung

Ein Passwort macht das Archiv durchgehend verschlüsselt: **Argon2id** leitet den Schlüssel ab, **XChaCha20-Poly1305** verschlüsselt den Inhalt, und **auch die Namen der Dateien werden verschlüsselt**. Ein falsches Passwort und ein manipuliertes Archiv werden beide erkannt — ein Archiv, das seine Prüfung nicht besteht, wird als beschädigt gemeldet, statt in Unsinn entpackt zu werden.

- `-p` fragt beim Erstellen zweimal nach dem Passwort, beim Lesen einmal; es wird nie angezeigt.
- Passwörter werden nie im Archiv gespeichert, und NikPak behält keines zwischen zwei Läufen.
- Dasselbe Passwort schützt die geteilten Volumes und die selbstextrahierende `.exe`.
- Es gibt keine Wiederherstellung: Ein verlorenes Passwort ist ein verlorenes Archiv — genau das ist der Sinn der Verschlüsselung.

5. Geteilte Volumes und selbstextrahierende Archive



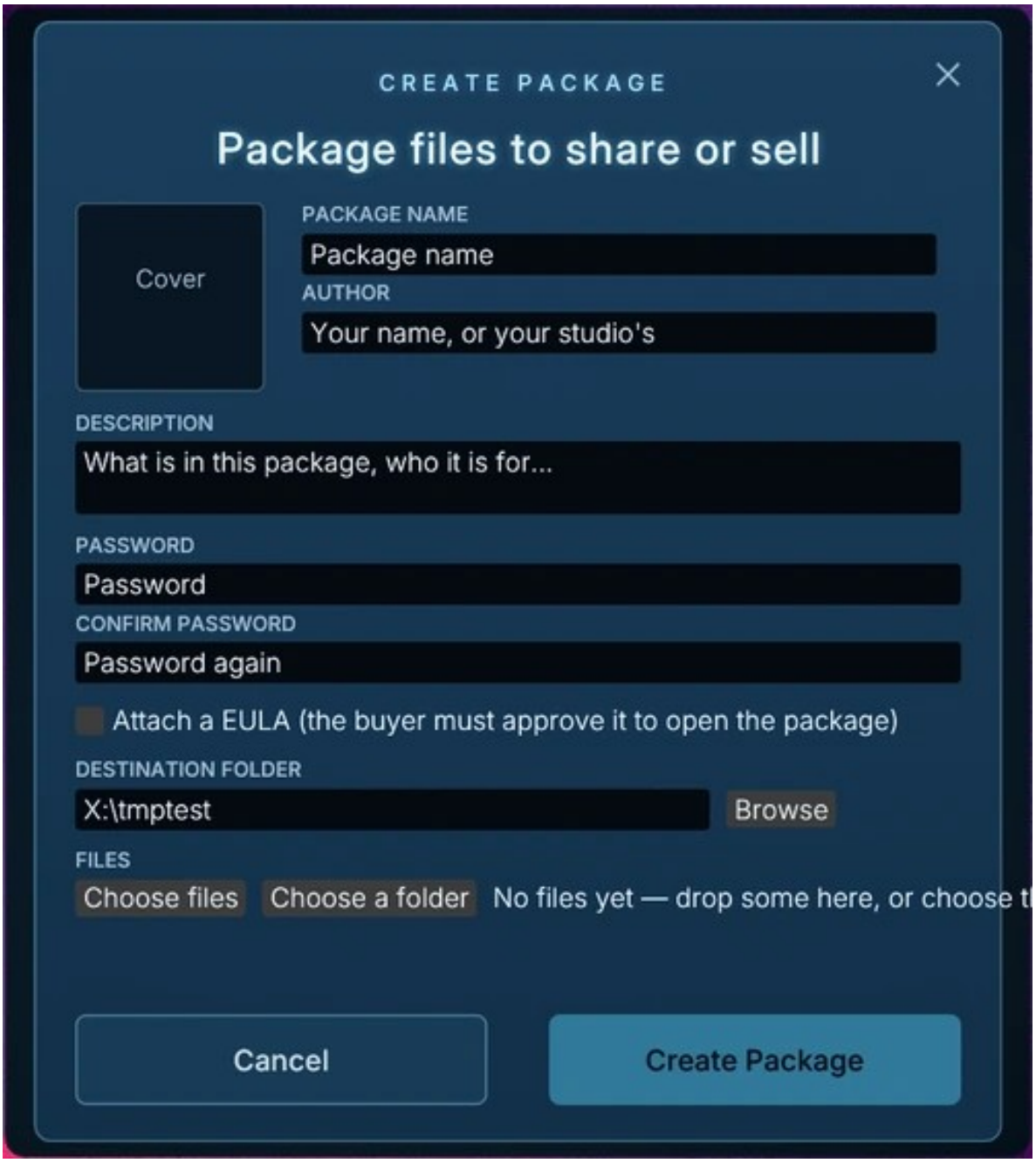
Das Teilen-Feld: Volume-Größen und die Option für selbstextrahierende Archive.

`-v` teilt ein Archiv in Volumes einer gewählten Größe — `nikpak a -v700m video.nik Movies\` schreibt `video.nik`, `video.n01`, `video.n02` … Zum Entpacken öffnen Sie das **erste** Volume, die übrigen werden automatisch gelesen; die anderen öffnet man nicht von Hand.

`-sfx` schreibt eine **selbstextrahierende `.exe`**: unNik mit angehängtem Archiv, sodass ein Empfänger sie ohne installierte Software entpackt. Es ist das Format für jemanden, der NikPak nicht hat, und es kann ein Passwort und eine rechtliche Erklärung tragen wie jedes andere Archiv.

6. Cover, rechtliche Erklärungen und Pakete

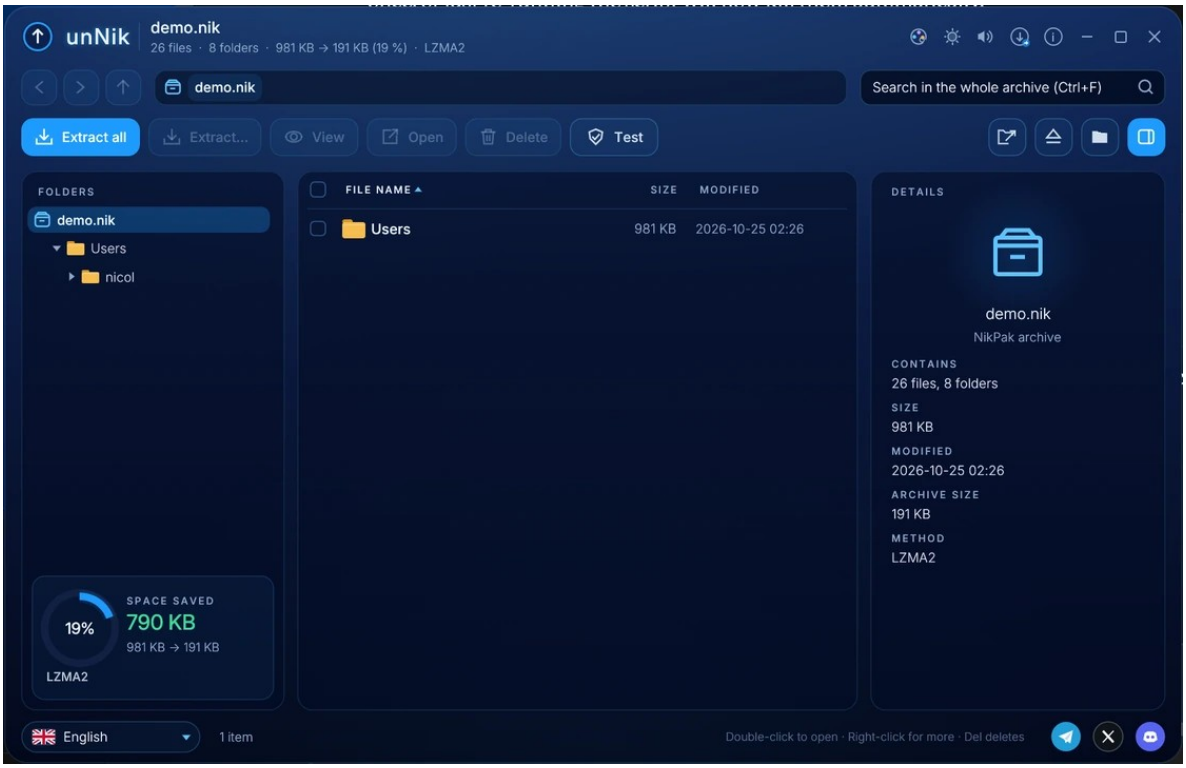
`-cover art.png` legt ein Bild bei — quadratisch, gespeichert mit 1024 × 1024 — und `-legal terms.txt` legt eine Erklärung bei. unNik zeigt beides, bevor es etwas entpackt, und nichts wird gelesen, bis der Leser zustimmt. Ein `cover.jpg` oder `terms.txt` neben den zu packenden Elementen wird automatisch übernommen.



Das Paket-erstellen-Feld: Name, Beschreibung, Cover und Lizenz.

Pakete führen denselben Gedanken weiter: ein `.nik`-Archiv mit einem Namen, einer Beschreibung, einem quadratischen Cover und einer Lizenz, die der Empfänger bestätigt — der NikPak-Zwilling einer Soundbank, gemacht zum Verkaufen oder Veröffentlichen. *Paket erstellen* und *Paket öffnen* stehen im Menü des Dateimanagers.

7. unNik, der kostenlose Browser

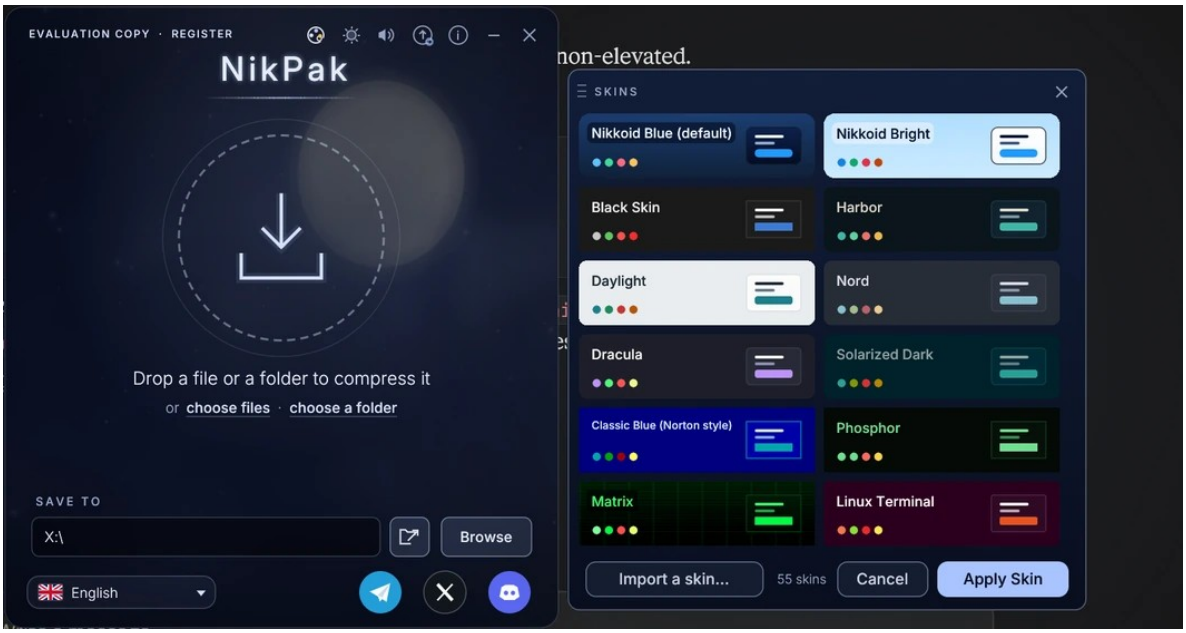


unNik beim Durchsuchen eines Archivs: Ordner links, Dateien in der Mitte, die Vorschau rechts.

unNik ist für alle kostenlos, für immer, ohne Registrierung — es ist das, was ein `.nik`-Archiv sicher zu versenden macht: Der Empfänger braucht nichts außer diesem Programm. Es öffnet ein Archiv in einem Fenster, durchsucht Ordner, sucht nach Namen, zeigt Text, Bilder und das Cover an und entpackt.

- **Erst ansehen, dann entpacken** — nichts wird auf die Festplatte geschrieben, bis Sie es verlangen.
- **Suchen** über die Dateinamen des Archivs, und sortieren nach Name, Größe, Datum oder Typ.
- **Anzeigen** von Text und Bildern aus dem Archiv, ohne sie auszupacken.
- **Entpacken** von allem, einem Ordner oder der Auswahl in einen Ordner Ihrer Wahl.
- **Nach dem Passwort fragen** nur bei Archiven, die eines haben; die übrigen öffnen sich still.

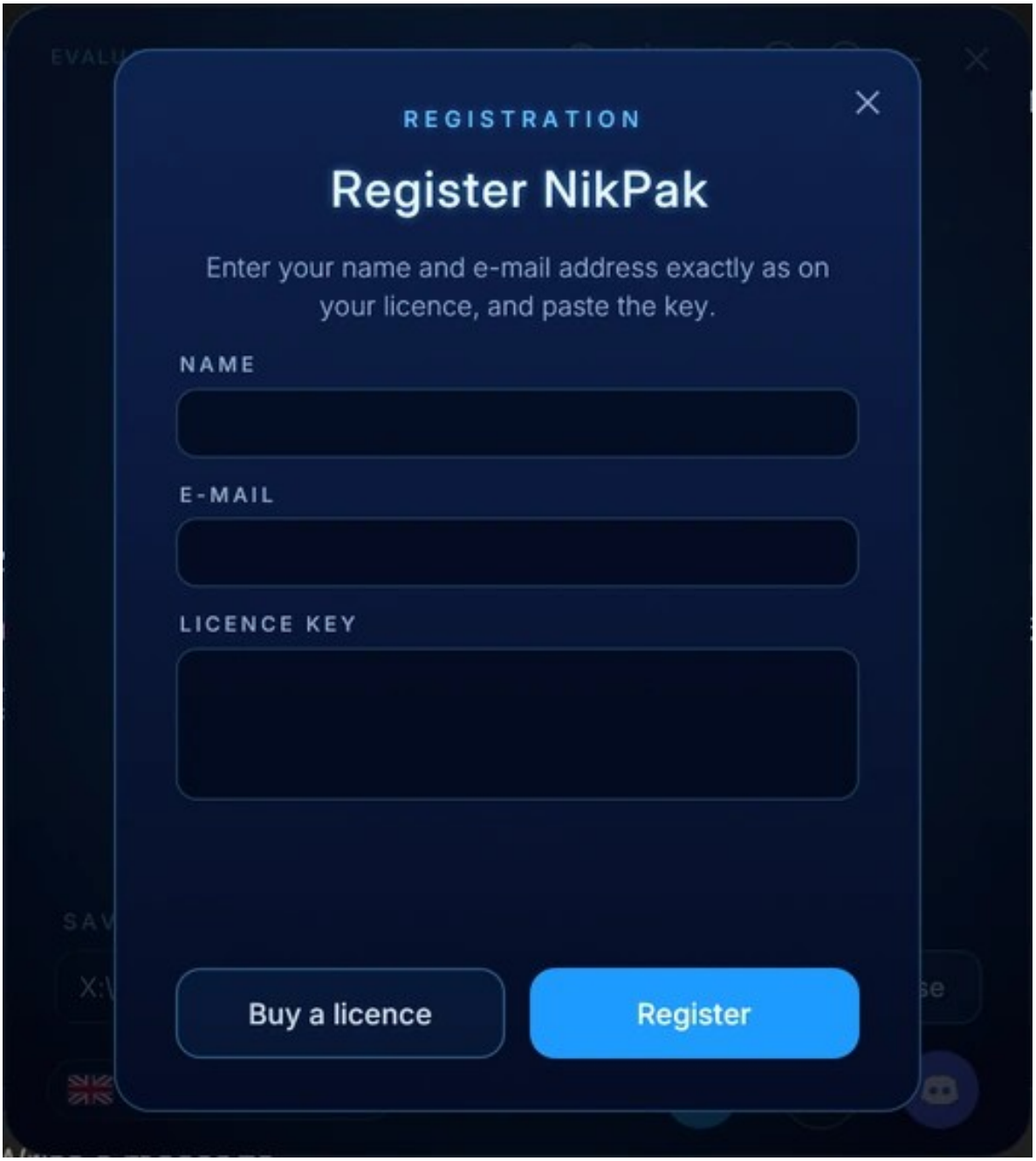
8. Skins, Sprachen und Registrierung



Das Skin-Feld neben dem Fenster: 89 Skins, gewählt mit Live-Vorschau.

89 Skins, von Amiga bis ZX Spectrum, mit Live-Vorschau neben dem Fenster. Zehn Oberflächensprachen — English, 中文, हिन्दी, Español, العربية, Français, □□□□□, Português, Русский und Bahasa Indonesia — gewählt mit der Flaggen-Schaltfläche.

nikpak ist Shareware: kostenlos für Privatpersonen, lizenziert pro Arbeitsplatz für Unternehmen. *Hilfe* ▸ *Registrieren* übernimmt Name, E-Mail und Schlüssel genau wie auf der Lizenz und prüft sie **offline**. unNik braucht überhaupt keine Lizenz.



Das Registrierungsfeld.

Tasten, die Sie brauchen

Taste	Aktion
`F1 · F10`	Hilfe · Skins
`Ctrl+O · Ctrl+N`	Archiv öffnen · neues Archiv
`Ctrl+E`	Entpacken
`Ctrl+T`	Archiv testen
`F5`	Aktualisieren
`Ctrl+Shift+L`	Oberflächensprache

Teil II · Technischer Leitfaden

Wie es gebaut ist, was es speichert und wie man es neu baut.

1. Architektur

NikPak ist ein Rust-Workspace aus zwei Binärdateien über einer Bibliothek: `nikpak` (der Archiver, Fenster und Kommandozeile) und `unNik` (der Browser). Die Bibliothek besitzt das Format, die Codecs und die Krypto, sodass die beiden Programme nicht uneinig darüber sein können, was ein Archiv bedeutet.

Die Bausteine

Teil	Was er tut
`Der Container`	Header, Blockindex, Kompressor pro Block, die verschlüsselte Namenstabelle
`Die Kompressoren`	Zstandard (schnelle Stufen), LZMA2 und PPMd (Maximum), pro Block
`Der Deflate-Rekompressor`	Dekodiert die inneren Streams von Office-Dateien, PDFs und gzip, rekomprimiert den Klartext-Inhalt, baut sie bitgenau wieder auf
`Die Medienfilter`	Probekompression für WAV, AIFF, MOD und BMP, das kleinste Ergebnis bleibt
`Der NAC-Audiocodec`	Ein verlustfreier Audiopfad für die Medienfilter
`Die Krypto`	Schlüsselableitung mit Argon2id, XChaCha20-Poly1305, authentifizierte Header
`unNik`	Dieselbe Bibliothek, schreibgeschützt, mit der Browser-Oberfläche

Die GUI ist egui, weshalb das Fenster eine einzige ausführbare Datei ist, ohne Laufzeitumgebung zum Installieren: Archiver und Browser kommen als einzelne Dateien, und die selbstextrahierende `.exe` ist unNik mit angehängtem Archiv.

2. Das .nik-Format

Ein Archiv besteht aus einem **Header**, einem **Blockindex**, den **Blöcken** und einer **verschlüsselten Namenstabelle**. Der Header trägt die Formatversion, die Flags, die Argon2id-Parameter und das Salt; der Index führt jeden Block mit seinem Kompressor, seinem Offset und seinen komprimierten und unkomprimierten Größen auf.

- **Blöcke sind unabhängig.** Das ermöglicht mehrthreadige Kompression und sorgt dafür, dass ein beschädigter Block eine Datei kostet statt das Archiv.
- **Die Namen stehen in einer eigenen Tabelle**, zusammen mit dem Inhalt verschlüsselt, sodass ein Archiv weder den Inhalt noch die Gestalt dessen preisgibt, was in ihm steckt.
- **Authentifiziert.** Header und Index sind vom selben Schlüssel abgedeckt, sodass ein manipulierter Index erkannt wird, bevor etwas geschrieben wird.
- **Versioniert.** Ein neueres Format wird mit einer Meldung abgelehnt, statt teilweise gelesen zu werden.
- **Volumes** sind derselbe Strom, an Blockgrenzen geschnitten, sodass jedes Volume das sein kann, das Sie öffnen, und die übrigen über den Namen gefunden werden.

3. Die Kompressions-Engine

Die Engine wählt einen Kompressor **pro Block** statt pro Archiv, weil die Daten in einem Ordner selten einen einzigen Charakter haben: Ein Foto und eine Quelldatei im selben Archiv sollten nicht durch denselben Pfad gezwungen werden. Auf der höchsten Stufe gehen Binärdaten zu LZMA2 und Text zu PPMd, und die Wahl wird im Blockindex festgehalten, sodass das Entpacken nichts raten muss.

Deflate-Rekompression ist die andere Hälfte des Größengewinns. Office-Dokumente, PDFs und `.gz`-Dateien sind Deflate-Streams in einem Container; NikPak dekodiert den inneren Stream, komprimiert den Klartext-Inhalt mit einem stärkeren Algorithmus und baut beim Entpacken den inneren Deflate-Stream **Byte für Byte** wieder auf, einschließlich seiner Blockgrenzen — sodass die Datei, die herauskommt, mit der identisch ist, die hineinging, und das Archiv viel kleiner ist als eines, das sie gespeichert hätte.

> Die Medienfilter arbeiten mit Probeläufen: WAV, AIFF, MOD und BMP werden mit dem allgemeinen Pfad und mit einem eigens gebauten Filter komprimiert (adaptive Vorhersage, Byte-Plane-Shuffling, ein Pixel-Prädiktor), und das **kleinste** Ergebnis bleibt. Der Filter kostet einen zweiten Durchlauf und bringt 12–15 % bei Audio und bis zu 38,8 % gegenüber RAR 7 bei Fotos.

4. Verschlüsselung und warum sie quantenresistent ist

Ein Passwort durchläuft **Argon2id** — speicherhart, sodass eine GPU oder ein ASIC wenig gewinnt — und der resultierende Schlüssel verschlüsselt mit **XChaCha20-Poly1305**, einer authentifizierten Stromchiffre mit 192-Bit-Nonce, die breit genug ist, um pro Archiv eine zufällige Nonce ohne Kollision zu wählen.

- **Nur symmetrische 256-Bit-Schlüssel.** Im Format gibt es kein RSA und keine elliptische Kurve, also auch keinen Public-Key-Austausch, den ein künftiger Quantencomputer brechen könnte: Die Sicherheit ruht auf der symmetrischen Chiffre und auf dem Passwort.
- **Authentifiziert, nicht nur verschlüsselt.** Poly1305 deckt den Inhalt ab; Header und Index werden mit demselben Schlüssel authentifiziert, sodass ein verändertes Archiv scheitert, statt zu etwas anderem zu dekodieren.
- **Dateinamen verschlüsselt.** Die Namenstabelle ist keine einfache Liste — ein Archiv verrät nicht, was es enthält.
- **Kein Wiederherstellungspfad**, konstruktionsbedingt: Nichts im Programm kann ein Archiv ohne sein Passwort entschlüsseln.

> Die Aussage ist bewusst eng: **Das Format hat keinen Public-Key-Schritt, den man brechen könnte**. Ein schwaches Passwort bleibt ein schwaches Passwort, weshalb die Schlüsselableitung speicherhart ist statt eines einfachen Hashes.

5. Benchmarks, Härtung und Build

Gegen RAR, 7-Zip und ZIP

Was die Tests gemessen haben

Korpus	Gegen RAR 7.00	Gegen 7-Zip 23.01	Gegen ZIP
Fotos	38,8 % kleiner	vorn	weit vorn
Audio (WAV, AIFF)	12–15 % kleiner	vorn	weit vorn
Office-Dokumente und PDFs	vorn, dank Deflate-Rekompression	vorn	weit vorn
Gemischte Ordner	vorn bei Maximum	gleichauf bei High	weit vorn
Geschwindigkeit auf den schnellen Stufen	schneller	gleichauf	langsamer als Store, konstruktionsbedingt

Die Zahlen stammen aus dem veröffentlichten Testkorpus, und die Skripte liegen im Repository, sodass sie sich erneut ausführen lassen, statt geglaubt werden zu müssen. Die ehrliche Zusammenfassung: Bei maximaler Kompression gewinnt NikPak bei Medien und bei Office-Dateien, ist bei gemischten Daten gleichauf mit 7-Zip, und versucht auf den schnellen Stufen kein Wettrennen um Geschwindigkeit zu gewinnen.

Härtung

- **Grenzengeprüftes Parsen.** Jede Länge in der Datei wird gegen die verbleibenden Bytes validiert, bevor sie verwendet wird; ein fehlerhaftes Archiv ist ein Fehler, kein Absturz.
- **Das Entpacken ist auf den Zielordner beschränkt**: Ein Eintrag namens `..\..\windows\system32` wird abgelehnt, nicht geschrieben.
- **Kein unsicherer Code in den Parsern**, wo er sich vermeiden lässt, und wo er sich nicht vermeiden lässt, sind die Invarianten neben dem Block dokumentiert.
- **Die Tests umfassen fehlerhafte Archive** — abgeschnittene Header, falsche Prüfsummen, ein Blockindex, der über das Ende hinauszeigt —, denn ein Format, das nur gültige Dateien sieht, ist ein Format, das nie getestet wurde.

Bauen

```
# Windows
.\build.ps1              # dist\nikpak.exe and dist\unNik.exe
.\build.ps1 -Test        # run the tests first
.\build.ps1 -Installer   # also the setup (needs Inno Setup 6)

# macOS / Linux
./build.sh               # dist/nikpak and dist/unNik

# command-line only, no window
cargo build --release --no-default-features
```

Die Release-Skripte halten **eine Version an einer Stelle**: `buildwin.ps1` schreibt die neue Nummer in `Cargo.toml`, `Cargo.lock`, `website/site.json` und `website/version.json`, bevor

kompiliert wird, und stellt sie wieder her, wenn der Build fehlschlägt; `buildmac.ps1` und `buildunix.ps1` lesen dieselbe `Cargo.toml` und verweigern den Build, wenn die gepushte Quelle eine andere Nummer trägt.

Das Skript jeder Plattform endet damit, `NikPak-<version>-<Platform>.zip` zu schreiben, mit dem Installationsprogramm, `unNik`, einer `README.txt` und den elf Handbüchern in einem Ordner `Documentation`, sodass die Website einen Download pro Plattform anbietet und nichts loses.

6. Dieses Buch

Die elf PDFs werden von `documents/text/<code>.py` durch `documents/build_docs.py` erzeugt, mit derselben Satz-Engine wie die übrigen Nikkoid-Produkte (`..\nkdocs\nkdocs.py`). Führen Sie `python documents/build_docs.py` nach einer Textänderung aus; `scripts\build-docs.ps1` erledigt das, bevor die ZIPs gepackt werden.